

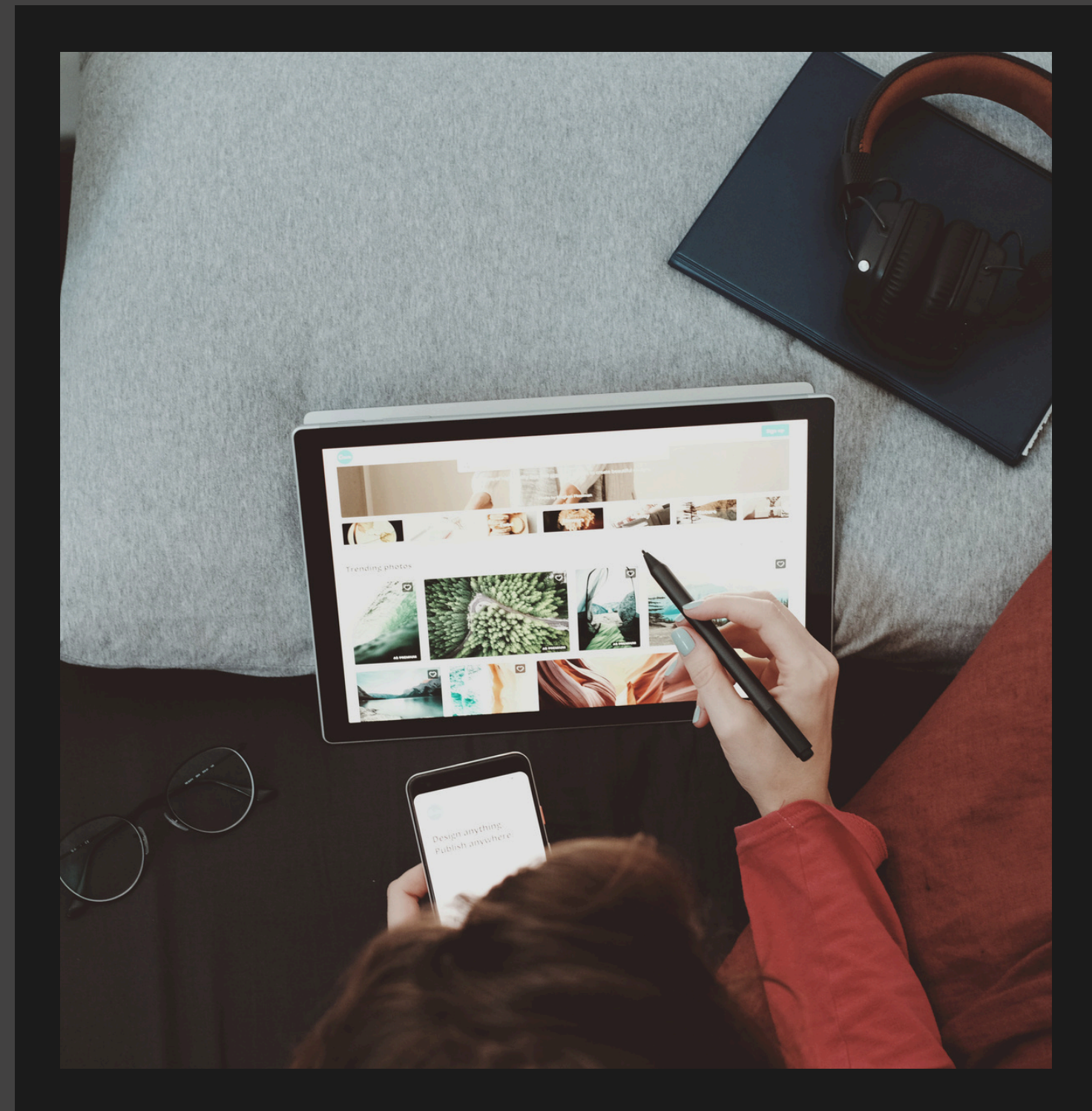
COMPARISON, HASH COLLISIONS, AND COMPLEXITY ANALYSIS

Data Structures & Algorithms

PREPARED BY: MAHMOUD ASHREY
DATA SCIENCE DIPLOMA

Overview of the Presentation

This presentation covers key data structures, hash collisions and their solutions, and an introduction to time complexity and its importance in algorithm efficiency.



Data Structures Comparison and Analysis

Data Structures Comparison Table

Comparison of access, search, insertion, and deletion times across common data structures.

Data Structure	Access	Search	Insert	Delete	Notes
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	Fixed size
Linked List	$O(n)$	$O(n)$	$O(n)$	$O(n)$	Dynamic
Stack	$O(n)$	$O(1)$	$O(1)$	$O(1)$	LIFO
Queue	$O(n)$	$O(1)$	$O(1)$	$O(1)$	FIFO
Hash Table	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$	Fast search
Binary Tree	$O(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	Sorted
Heap	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	Priority



What is a Data Structure?

A data structure is a way of organizing and storing data efficiently.

It helps in accessing, modifying, and managing data.

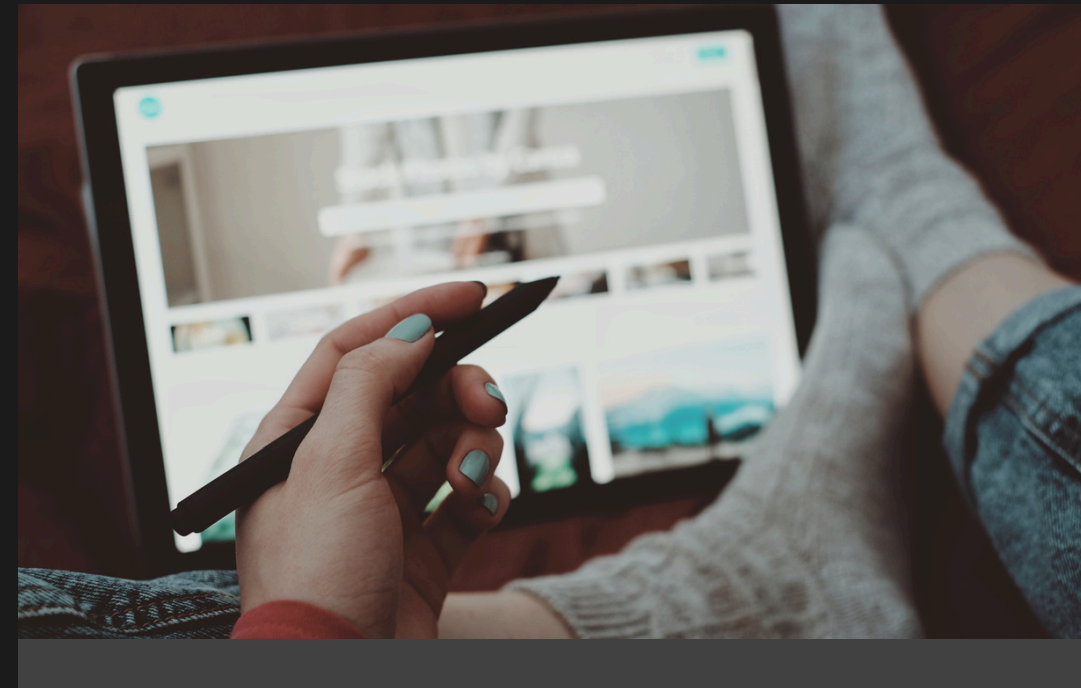
Types of Data Structures



LINEAR DATA STRUCTURES

Data elements are arranged sequentially.

Examples:
Array, Linked List, Stack, Queue



NON-LINEAR DATA STRUCTURES

Data elements are not in sequence.

Examples:
Tree, Heap, Hash Table, Graph

Choosing the right data
structure can significantly
improve algorithm
performance.

— Computer Science Principle

Array

1

DEFINITION

Stores elements in contiguous memory locations.

2

ADVANTAGES

Fast access using index ($O(1)$).

3

DISADVANTAGES

Fixed size and slow insertion ($O(n)$).

Linked List

1

DEFINITION

Elements are connected using nodes and pointers.

2

ADVANTAGES

Dynamic size and efficient insertion ($O(1)$).

3

DISADVANTAGES

Slow access ($O(n)$) and extra memory for pointers.

Stack

1

DEFINITION

Follows LIFO (Last In First Out).

2

ADVANTAGES

Fast insertion and deletion ($O(1)$).

3

DISADVANTAGES

Limited access (only top element).

Queue

1

DEFINITION

Follows FIFO (First In First Out).

2

ADVANTAGES

Efficient insertion and deletion ($O(1)$).

3

DISADVANTAGES

Limited access (only front element).

Hash Table

1

DEFINITION

Stores data as key-value pairs using a hash function.

2

ADVANTAGES

Very fast search, insertion, and deletion ($O(1)$).

3

DISADVANTAGES

May cause collisions and depends on hash function.

Tree

1

DEFINITION

A hierarchical structure with nodes connected by edges.

2

ADVANTAGES

Efficient searching and sorting ($O(\log n)$).

3

DISADVANTAGES

Complex implementation and slower if unbalanced.

Heap

1

DEFINITION

A special tree where parent nodes follow a specific order.

2

ADVANTAGES

Efficient for finding min or max values ($O(1)$).

3

DISADVANTAGES

Not efficient for searching arbitrary elements.

Hash Collision and Solutions



What is Hash Collision?

A collision occurs when two different keys produce the same hash value.

This causes both elements to be stored in the same index in a hash table.

Collision Resolution Methods

There are two main approaches to handle collisions:

1. Chaining
2. Open Addressing

Open Addressing includes:

- Linear Probing
- Quadratic Probing
- Double Hashing

Chaining

**STORES MULTIPLE
ELEMENTS AT THE SAME
INDEX USING A LINKED LIST.**

Example:

Index 5 → Key1 → Key2 → Key3

ADVANTAGES:

Easy to implement and allows multiple elements per index.

Linear Probing

**IF A POSITION IS OCCUPIED,
MOVE TO THE NEXT
AVAILABLE SLOT.**

Example:

Index 5 → occupied

Index 6 → store element

DISADVANTAGES:

May cause clustering (many elements grouped together).

Time Complexity Analysis



What is Time Complexity?

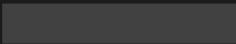
Time complexity measures how fast an algorithm runs as the input size increases.

It helps us evaluate the efficiency of algorithms.

order of growth	name	typical code framework	description	example
1	constant	<code>a = b + c;</code>	statement	add two numbers
$\log N$	logarithmic	<code>while (N > 1) { N = N / 2; ... }</code>	divide in half	binary search
N	linear	<code>for (int i = 0; i < N; i++) { ... }</code>	loop	find the maximum
$N \log N$	linearithmic	[see mergesort lecture]	divide and conquer	mergesort
N^2	quadratic	<code>for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) { ... }</code>	double loop	check all pairs
N^3	cubic	<code>for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) for (int k = 0; k < N; k++) { ... }</code>	triple loop	check all triples
2^N	exponential	[see combinatorial search lecture]	exhaustive search	check all subsets

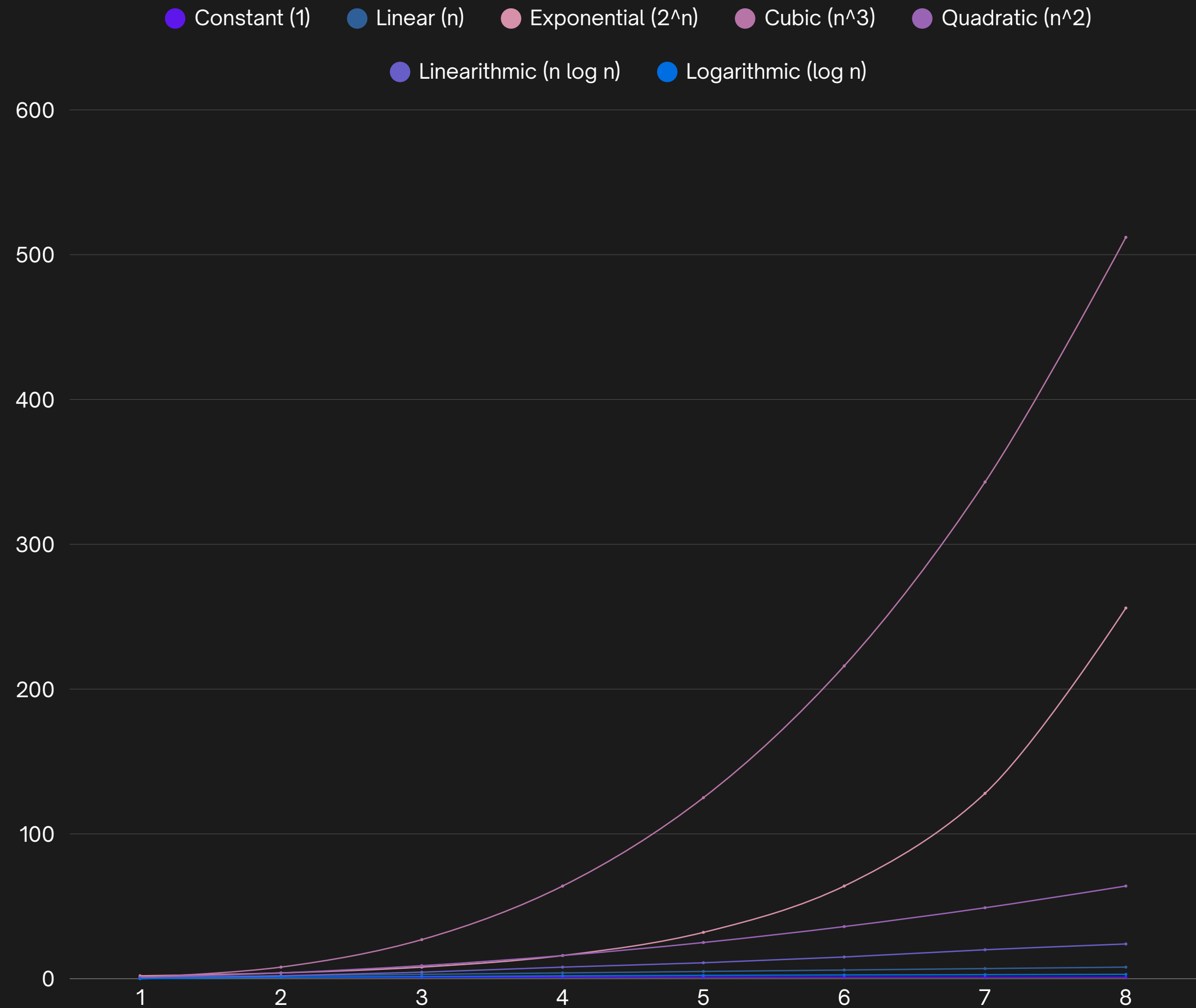
Complexity Growth Comparison

This chart shows how different algorithms grow as input size increases.



As input size increases, algorithms with higher complexity become slower.

Lower complexity means better performance as input size increases.



Thank You